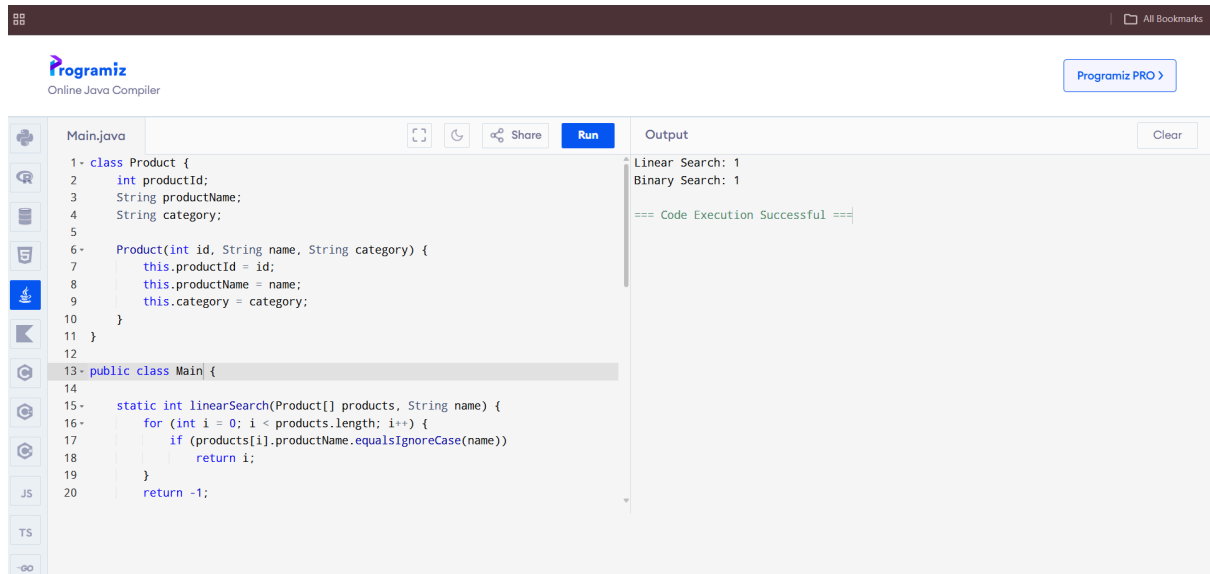


DSA

2. E-commerce Platform Search Function

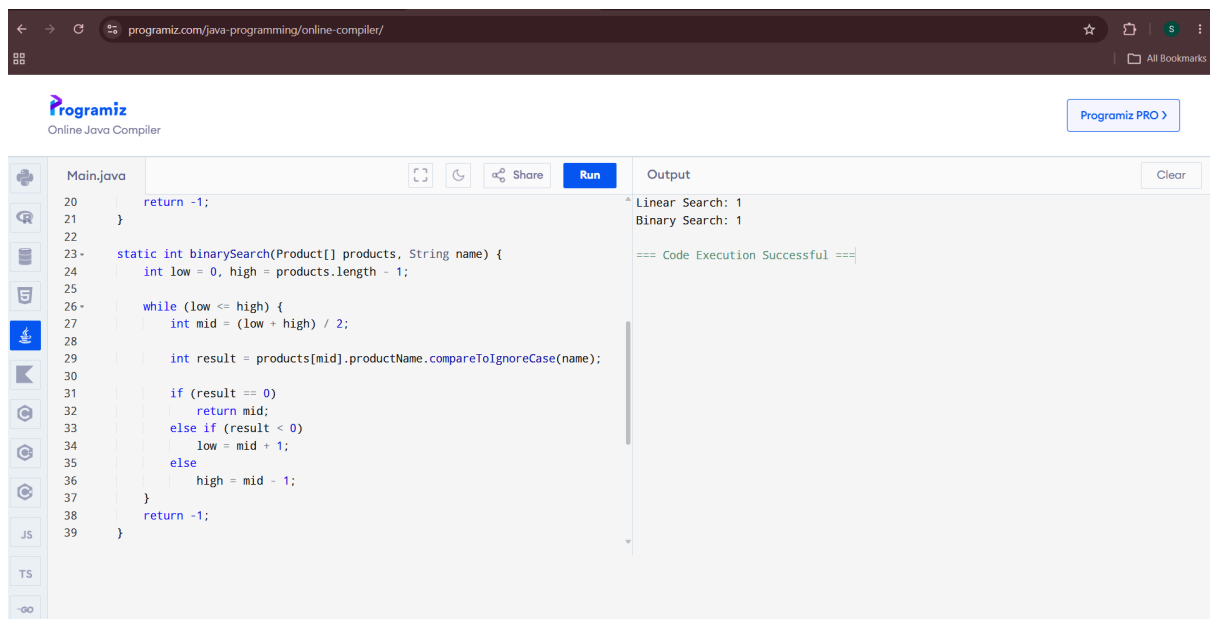


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
1- class Product {
2-     int productId;
3-     String productName;
4-     String category;
5-
6-     Product(int id, String name, String category) {
7-         this.productId = id;
8-         this.productName = name;
9-         this.category = category;
10-    }
11- }
12-
13- public class Main {
14-
15-     static int linearSearch(Product[] products, String name) {
16-         for (int i = 0; i < products.length; i++) {
17-             if (products[i].productName.equalsIgnoreCase(name))
18-                 return i;
19-         }
20-         return -1;
21-     }
22- }
```

The output window displays the following text:

```
Linear Search: 1
Binary Search: 1
=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface with the code editor displaying a binary search implementation:

```
20-     return -1;
21- }
22-
23- static int binarySearch(Product[] products, String name) {
24-     int low = 0, high = products.length - 1;
25-
26-     while (low <= high) {
27-         int mid = (low + high) / 2;
28-
29-         int result = products[mid].productName.compareToIgnoreCase(name);
30-
31-         if (result == 0)
32-             return mid;
33-         else if (result < 0)
34-             low = mid + 1;
35-         else
36-             high = mid - 1;
37-     }
38-     return -1;
39- }
```

The output window displays the following text:

```
Linear Search: 1
Binary Search: 1
=== Code Execution Successful ===
```

The screenshot shows the Programiz Online Java Compiler interface. The code editor contains a Java program named `Main.java` that implements a binary search algorithm. The code defines a `Product` class with attributes `id`, `name`, and `category`. It then creates an array of `Product` objects and uses `linearSearch` and `binarySearch` methods to find the index of a "Laptop" in the array. The output window shows the results of the searches: "Linear Search: 1" and "Binary Search: 1", followed by a success message.

```
35         else
36             high = mid - 1;
37     }
38     return -1;
39 }
40
41 public static void main(String[] args) {
42     Product[] products = {
43         new Product(1, "Keyboard", "Electronics"),
44         new Product(2, "Laptop", "Electronics"),
45         new Product(3, "Mouse", "Electronics")
46     };
47
48     System.out.println("Linear Search: " +
49         linearSearch(products, "Laptop"));
50
51     System.out.println("Binary Search: " +
52         binarySearch(products, "Laptop"));
53 }
54 }
```

Output:

```
Linear Search: 1
Binary Search: 1

=== Code Execution Successful ===
```

7. Financial Forecasting (Recursion)

The screenshot shows the Programiz Online Java Compiler interface. The code editor contains a Java program named `FinancialForecast.java` that implements a recursive method to calculate the future value of an investment. The `futureValue` method takes the current value, growth rate, and number of years as input and returns the future value. The `main` method initializes the variables and calls the `futureValue` method. The output window shows the result: "Future Value = 16105.100000000008", followed by a success message.

```
1 public class FinancialForecast {
2
3     static double futureValue(double currentValue,
4                               double growthRate,
5                               int years) {
6
7         if (years == 0)
8             return currentValue;
9
10        return futureValue(
11            currentValue * (1 + growthRate),
12            growthRate,
13            years - 1);
14    }
15
16    public static void main(String[] args) {
17
18        double currentValue = 10000;
19        double growthRate = 0.10;
20        int years = 5;
21    }
```

Output:

```
Future Value = 16105.100000000008

=== Code Execution Successful ===
```



FinancialForecast.java

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

```
        return currentValue;
    }

    return futureValue(
        currentValue * (1 + growthRate),
        growthRate,
        years - 1);
}

public static void main(String[] args) {
    double currentValue = 10000;
    double growthRate = 0.10;
    int years = 5;

    double result =
        futureValue(currentValue, growthRate, years);

    System.out.println("Future Value = " + result);
}
```

Output

Future Value = 16105.100000000008

=== Code Execution Successful ===

Clear

JS

TS

-∞